FAULTGUARD AI

Knowledge-Driven IoT Fault Diagnosis Assistant

PROJECT REPORT

ABES Engineering College
Department of Computer Science & Engineering

Submitted by: Akansha Kukshal
Enrollment Number: 2400320100117
Project Supervisor: Raveendra Reddy
Academic Year: 2026–2027

Prepared for HCL Career Shaper

Document Status and Verification Note

This report was initially prepared using the project source code and SRS. Subsequent manual testing was performed on 30 October 2026. HiveMQ connectivity, MQTT publishing and subscription, subscriber-side ML inference, rule-based diagnosis, PostgreSQL connectivity and record retrieval, and dashboard display were observed successfully. Dashboard values changed upon refresh, but the exact source of every displayed value was not independently traced. Model metrics have not been independently regenerated. Physical ESP32 hardware deployment and automated unit testing have not been verified.

Contents

- Abstract
- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Scope and Limitations
- 5. System Architecture
- 6. Technology Stack
- 7. Dataset and Processing
- 8. Machine Learning
- 9. Diagnosis Logic
- 10. Database and Dashboard
- 11. Model Results
- 12. Testing and Verification
- 13. Security and Dependencies
- 14. Setup and Execution
- 15. Future Enhancements
- 16. Conclusion
- References

Abstract

FaultGuard AI is a software-based industrial monitoring and fault-diagnosis prototype. It processes sensor observations, classifies them as normal or anomalous using a Random Forest model and StandardScaler, interprets abnormal sensor patterns using configured ranges, and presents results through a Streamlit dashboard. The supplied implementation uses SKAB Valve 1 data, MQTT messaging through HiveMQ Cloud, PostgreSQL storage, and Plotly visualizations. Physical ESP32 hardware deployment is not verified.

1. Introduction

Industrial equipment generates readings such as vibration, temperature, pressure, current, voltage and flow rate. FaultGuard AI demonstrates a modular workflow for processing sensor records, flagging abnormal patterns, retaining historical readings, and presenting the result to a monitoring user.

2. Problem Statement

Manual review of multiple sensor streams can make it difficult to consistently identify abnormal operating patterns and review historical behavior. This project combines dataset-driven sensor input, anomaly classification, rule-based interpretation, database storage and dashboard visualization. It does not directly control machinery or guarantee prediction of every future failure.

3. Objectives

• Process eight sensor features: Accelerometer1RMS, Accelerometer2RMS, Current, Pressure, Temperature, Thermocouple, Voltage, and Volume Flow RateRMS.

• Train and use a Random Forest classifier for NORMAL/ANOMALY classification.

• Use StandardScaler consistently at training and inference.

• Support MQTT publish/subscribe through HiveMQ Cloud.

• Generate possible causes, severity and recommended inspection actions.

• Store readings in PostgreSQL and visualize results in Streamlit.

4. Scope and Limitations

In scope: dataset-driven simulation, MQTT messaging, anomaly classification, rule-based diagnosis, PostgreSQL storage and dashboard visualization. Out of scope: physical machinery control, automatic repair and safety-critical autonomous decisions. The classifier is binary; cause descriptions are derived from sensor-range rules rather than a verified multi-class fault classifier.

5. System Architecture

Data flow based on the uploaded code and README:

SKAB Valve 1 CSV → sensor_simulator.py → mqtt_publisher.py → HiveMQ Cloud → mqtt_subscriber.py → StandardScaler + Random Forest → rule-based cause/severity/recommendation → PostgreSQL and latest_data.json → Streamlit dashboard.

The SRS includes optional hardware variants, but physical ESP32 deployment is not verified.

6. Technology Stack

Python; Pandas and NumPy; scikit-learn RandomForestClassifier and StandardScaler; Joblib; Paho MQTT; HiveMQ Cloud; PostgreSQL with psycopg2; Streamlit; Plotly; JSON and CSV; Git/GitHub.

7. Dataset and Processing

The supplied README and simulator identify SKAB Valve 1 files under archive/SKAB/valve1. The simulator reads semicolon-separated CSV files and maps eight sensor features into messages. The training script sorts records by datetime, fills missing features with medians, applies a chronological 80/20 split, scales features and trains a Random Forest model. Training was not independently rerun for this report.

8. Machine Learning

The model uses RandomForestClassifier with 200 estimators, random_state=42 and class_weight=balanced. StandardScaler standardizes the eight features. fault_detector.py loads fault_model.joblib and scaler.joblib, applies the scaler and returns NORMAL/ANOMALY plus normal and anomaly probabilities. Probabilities are model outputs, not guarantees about physical safety.

9. Diagnosis Logic

The MQTT subscriber calls the model and, for anomalies, compares readings against configured normal ranges. Out-of-range features are grouped into possible pressure, flow, mechanical vibration, electrical or thermal abnormalities. If no single range explains an anomaly, the system reports an abnormal multivariate pattern. The subscriber attempts to store records in PostgreSQL and writes the latest dashboard payload to latest_data.json; successful runtime execution has not been independently verified.

10. Database and Dashboard

database.py uses psycopg2 and reads POSTGRES_PASSWORD from environment variables. Defaults are localhost:5432 and database faultdiagnosis_db. The subscriber inserts sensor and diagnosis fields into sensor_readings. dashboard.py displays machine status, key indicators, sensor readings, trends, anomaly probability, possible cause and recommended action. It initially requires latest_data.json and uses historical_data.csv as a fallback for historical charts.

11. Model Results

The following metrics are copied from the uploaded model_metrics.json and README; they have not been independently regenerated. Accuracy: 93.17%. Normal precision: 90.69%; normal recall: 99.92%; normal F1: 95.08%. Anomaly precision: 99.80%; anomaly recall: 80.08%; anomaly F1: 88.86%. Confusion matrix (actual rows, predicted columns): actual normal [2396 normal, 2 anomaly]; actual anomaly [246 normal, 989 anomaly]. The model missed 246 anomalous examples in the reported evaluation, so recall is a material limitation.

12. Testing and Verification

The code was not executed end-to-end while preparing this report. The following are proposed test cases, not claims of completed tests.

ID	Test case	Expected result	Status
T-01	Python dependencies	Required packages installed in virtual environment	Passed
T-02	HiveMQ connection	Successful connection to HiveMQ Cloud	Passed
T-03	MQTT publisher	Sensor messages published successfully	Passed
T-04	MQTT subscriber	Messages received on configured topic	Passed
T-05	ML inference	Anomaly prediction and probability displayed	Passed
T-06	Rule-based diagnosis	Severity, possible cause and recommendation generated	Passed
T-07	PostgreSQL connection	Database connection successful	Passed
T-08	Database storage	Records saved and retrieved successfully	Passed
T-09	Dashboard display	Sensor values and machine status displayed	Passed
T-10	Dashboard refresh	Changing values observed after refresh	Observed; source not fully traced
T-11	Model retraining	Not performed	Not tested

13. Security and Dependencies

The project uses environment variables for MQTT and database credentials. The updated requirements.txt contains Streamlit, pandas, NumPy, scikit-learn, joblib, Plotly, psycopg2-binary, paho-mqtt and python-dotenv. The .env file is excluded from Git using .gitignore and must not be uploaded to the repository.

14. Setup and Execution

1. Create and activate a Python virtual environment.

2. Install requirements after adding paho-mqtt and python-dotenv.

3. Place SKAB Valve 1 CSV files under archive/SKAB/valve1.

4. Create a local .env containing HIVEMQ_USERNAME, HIVEMQ_PASSWORD and POSTGRES_PASSWORD; do not upload it.

5. Configure PostgreSQL database faultdiagnosis_db and a sensor_readings table matching the subscriber INSERT.

6. Run python mqtt_subscriber.py, then python mqtt_publisher.py in separate terminals.

7. Run streamlit run dashboard.py and ensure required latest_data.json and historical_data.csv files are available.

These are inferred setup steps; successful end-to-end operation is not verified.

15. Future Enhancements

Add repeatable unit and integration tests; document a database schema; improve input validation, reconnect handling and error logging; make diagnosis thresholds configurable; expand labelled fault classes only when appropriate labelled data is available; and record a verified demo video.

16. Conclusion

FaultGuard AI demonstrates a modular prototype combining SKAB-derived sensor data, Random Forest anomaly detection, StandardScaler, MQTT, rule-based diagnosis, PostgreSQL and Streamlit. Existing project artifacts report 93.17% accuracy and 80.08% anomaly recall. Before submission, run the system in a clean environment, address missing dependency declarations, document the database schema and capture actual test evidence.

References

1. HCLTech, Project Creation Guide — Embedded Systems & IoT, Version 1.0, September 2026.

2. Student-supplied SRS: Knowledge-Driven IoT Fault Diagnosis Assistant.

3. SKAB Valve 1 dataset referenced by the project.

4. Documentation for Python, pandas, scikit-learn, Streamlit, Paho MQTT, PostgreSQL/psycopg2, Plotly and Joblib.